

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and
Commerce Department of Information Technology (B.Sc.I.T
Semester IV) AI(Artificial Intelligence)
Practical VII (VI & VII & VIII)

Roll No.:T052	Name:Divyanjali yadav
Class:TYIT	Batch:01
Date of Assignment:7-08-2026	Date/Time of Submission:7-08-2026

6. Implement Decision Tree regression on a regression dataset.

Code:import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeRegressor

data = pd.read_csv("iris (2).csv")

x = data[['sepal_length']]

y = data['petal_length']

x_train, x_test, y_train, y_test = train_test_split(

x,

y,

test_size=0.3,

random_state=1

)

model =

DecisionTreeRegressor()

model.fit(x_train, y_train)

print("Training Accuracy:", model.score(x_train, y_train) * 100)

print("Testing Accuracy:", model.score(x_test, y_test) * 100)

new_data = pd.DataFrame([[5.5]], columns=['sepal_length'])

prediction = model.predict(new_data)

print("Predicted Petal Length for sepal length 5.5:", prediction[0])

print("Divyanjali yadav")

```
Training Accuracy: 86.44982464055326
Testing Accuracy: 72.01267791587007
Predicted Petal Length for sepal length 5.5: 3.2
Divyanjali yadav
```

7. Implement k-Nearest Neighbors (kNN) classifier.

Code:from sklearn.datasets import load_iris

from sklearn.model_selection import train_test_split

from sklearn.neighbors import KNeighborsClassifier

iris = load_iris()

X = iris.data

y = iris.target

X_train, X_test, y_train, y_test = train_test_split(

X,

y,

test_size = 0.3,

random_state = 1

)

model = KNeighborsClassifier(n_neighbors = 3)

model.fit(X_train, y_train)

```
Training Accuracy: 95.23809523809523
Testing Accuracy: 97.77777777777777
Predicted Class: 0
Divyanjali yadav
```

```

model.fit(X_train,y_train)
print("Trainning Accuracy:",model.score(X_train,y_train)*100)
print("Testing Accuracy:",model.score(X_test,y_test)*100)
prediction = model.predict([X_test[0]])
print("Predicted Class:",prediction[0])
print("Divyanjali yadav")

```

8. Implement multiclass classification using Logistic Regression or Decision

Tree.

```

from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split
from sklearn.tree import DecisionTreeClassifier
iris = load_iris()
X = iris.data
y = iris.target
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X, y,
    test_size=0.3,
    random_state=1
)
# Create Decision Tree model
model =
DecisionTreeClassifier()
# Train model
model.fit(X_train, y_train)
# Display accuracy
print("Training Accuracy:", model.score(X_train, y_train)*100)
print("Testing Accuracy:", model.score(X_test, y_test)*100)
# Predict one sample
prediction = model.predict([X_test[0]])
print("Predicted Class:", prediction[0])
print("Divyanjali yadav")

```

```

Training Accuracy: 100.0
Testing Accuracy: 95.55555555555556
Predicted Class: 0
Divyanjali yadav

```

Perform Logistic Regression, Decision Tree, and kNN on the same dataset. Compare the Results.

```

import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import accuracy_score
# Load dataset
data = pd.read_csv("world_cup_results.csv")
# Create Result column
data['Result'] = data.apply(
    lambda row: 'Home Win' if row['HomeGoals'] > row['AwayGoals']
    else ('Away Win' if row['HomeGoals'] < row['AwayGoals'] else 'Draw'),
    axis=1
)
# Convert teams into numbers
encoder = LabelEncoder()

```

```

data['HomeTeam'] = encoder.fit_transform(data['HomeTeam'])
data['AwayTeam'] = encoder.fit_transform(data['AwayTeam'])
# Features and target
X = data[['Year', 'HomeTeam', 'AwayTeam']]
y = data['Result']
# Split dataset
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.3,
    random_state=1
)
# 1. Logistic Regression
logistic_model = LogisticRegression(max_iter=1000)
logistic_model.fit(X_train, y_train)
logistic_pred = logistic_model.predict(X_test)
logistic_accuracy = accuracy_score(
    y_test,
    logistic_pred
) * 100
# 2. Decision Tree
tree_model = DecisionTreeClassifier(random_state=1)
tree_model.fit(X_train, y_train)
tree_pred = tree_model.predict(X_test)
tree_accuracy = accuracy_score(
    y_test,
    tree_pred
) * 100
# 3. kNN
knn_model = KNeighborsClassifier(n_neighbors=5)
knn_model.fit(X_train, y_train)
knn_pred = knn_model.predict(X_test)
knn_accuracy = accuracy_score(
    y_test,
    knn_pred
) * 100
# Compare Results
print("Logistic Regression Accuracy:",
      logistic_accuracy, "%")
print("Decision Tree Accuracy:",
      tree_accuracy, "%")
print("kNN Accuracy:",
      knn_accuracy, "%")
# Find best model
if logistic_accuracy >= tree_accuracy and logistic_accuracy >= knn_accuracy:
    print("Best Model: Logistic Regression")
elif tree_accuracy >= logistic_accuracy and tree_accuracy >= knn_accuracy:
    print("Best Model: Decision Tree")
else:
    print("Best Model: kNN")

```

```

Logistic Regression Accuracy: 53.90625 %
Decision Tree Accuracy: 48.828125 %
kNN Accuracy: 51.171875 %
Best Model: Logistic Regression

```